

4 How can we estimate the spectral norm of a matrix more scientifically?

May By Su Jianlin | 2026-05-04 | 1762 readers | [Kimi \(Citation,](#)

The spectral norm is a core concept in matrix analysis and plays a crucial role in deep learning as well. From the Lipschitz constraints required in the WGAN era to the training stability of LLM today, and the burgeoning Muon optimizer, all are closely related to the spectral norm of matrix parameters. Therefore, how to efficiently and accurately estimate the spectral norm is becoming increasingly important and deserves our in-depth study.

As is well known, power iteration is the standard approach for estimating spectral norms, but it still has considerable room for improvement. This article will briefly summarize some approaches to estimating spectral norms, including improving the convergence speed of power iteration and how to estimate the strict upper bound of the spectral norm, etc.

Spectral norm

The definition of spectral norm is

$$\|W\|_2 = \max_{\|x\|_2=1} \|Wx\|_2 \quad (1)$$

in $W \in \mathbb{R}^{n \times m}$, $x \in \mathbb{R}^m$. Without loss of generality, let's assume $n \geq m$. By definition, the spectral norm represents the "expansion rate" of a linear layer—the magnitude of the output vector increases at most after the input vector passes through the linear layer. $\|W\|_2$ The spectral norm is also equal to the largest singular value of the matrix (for proof, refer to "The Road to Low-Rank Approximation (Part Two): SVD"). We will not elaborate on these basic concepts further.

My first encounter with spectral norms was during the era of GAN popularity, when WGAN emerged, emphasizing the necessity for the discriminator to satisfy the Lipschitz constraint. A linear layer... Wx The Lipschitz constant is precisely the matrix W . The spectral norm of spectral learning leads to techniques such as spectral normalization and spectral regularization. Interested students can refer to "Lipschitz Constraints in Deep Learning: Generalization and Generative Models".

In recent years, the increasingly popular Muon optimizer is also closely related to the spectral norm, which is often considered the steepest descent under the spectral norm. Furthermore, in "Beyond MuP: 4. Upholding Parameter Stability," we also propose constraining the spectral norm of the parameters to ensure training stability. All of these aspects necessitate the accurate calculation of the spectral norm.

Power iteration

Calculating the spectral norm using SVD is the most direct method, but it is obviously too expensive. We usually use power iteration.

$$\mathbf{v}^{(t)} = \frac{\mathbf{W}^\top \mathbf{W} \mathbf{v}^{(t-1)}}{\|\mathbf{W}^\top \mathbf{W} \mathbf{v}^{(t-1)}\|_2} \quad (2)$$

It is $(\sigma_2/\sigma_1)^2$. The velocity converges to the right principal singular vector $\mathbf{v}_1$. After one iteration, we have

$$\sigma_1 \approx \|\mathbf{W} \mathbf{v}^{(t)}\|_2 \quad (3)$$

The proof can be found in "Reflections on the Novel Weight Decay from Spectral Norm Gradient". In practice, if we only care about the spectral norm and not the singular vectors, the convergence speed is often faster than... $(\sigma_2/\sigma_1)^2$. More optimistic. Furthermore, the results of power iteration are easily affected by the initial value. $\mathbf{v}^{(0)}$. Fluctuations caused by the influence of [the environment] can be addressed by considering parallel computing. Find the path, and then find the maximum value.

If calculated simultaneously. The path, and in each iteration, replaces L2 Normalize with this. Orthogonalization of vectors, i.e.

$$\mathbf{V}^{(t)} = \text{QR}(\mathbf{W}^\top \mathbf{W} \mathbf{V}^{(t-1)}) \quad (4)$$

Then the result will converge to the previous... Given a right singular vector, the first... The principle behind the singular value can be found in "Muon Implementation Based on Streaming Power Iteration: 4. Principle". However, this article will not delve into this extension, but will focus on the estimation of the spectral norm—that is, the maximum singular value.

Find the gradient .

First, here is a sample implementation of exponentiation based on Jax:

```

1  import jax, jax.lax as lax
2  import jax.numpy as jnp
3
4  @jax.jit(static_argnums=(1,))
5  def l2_normalize(x, axis=-2):
6      return x / jnp.linalg.vector_norm(x, axis=axis, keepdims=True)
7
8  @jax.jit(static_argnums=(1, 2, 3))
9  def spec_norm_v1(w, T=10, k=1, key=42):
10     v_shape = w.shape[:-2] + w.shape[-1:] + (k,)
11     v_init = jax.random.normal(jax.random.PRNGKey(key), v_shape)
12     v_step = lambda i, v: l2_normalize(w.mT @ (w @ v))
13     v = lax.fori_loop(0, T, v_step, v_init)
14     return jnp.linalg.vector_norm(w @ v, axis=-2).max(axis=-1)

```

Clearly, the time complexity of exponentiation iteration is $O(n)$. $O(mnT)$. From a complexity perspective, this is already quite ideal. If we are only using it for monitoring purposes, then the above approach is sufficient. However,

r, if we are using it for spectral normalization or spectral regularization, then we need the gradient of the spectral norm. According to the above approach, it needs to propagate backward step by step along the power iteration loop, which results in relatively high computational complexity.

In fact, we already derived the gradient of the spectral norm in "Thoughts on the Novel Weight Decay from the Gradient of the Spectral Norm". $\nabla_{\mathbf{W}} \sigma_1 = \mathbf{u}_1 \mathbf{v}_1^\top$ Therefore, we can directly define its gradient as follows:

$\nabla_{\mathbf{W}} \sigma_1 = \mathbf{u}_1 \mathbf{v}_1^\top$ This reduces the complexity of backpropagation. In addition, we can also utilize... $\sigma_1 = \mathbf{u}_1^\top \mathbf{W} \mathbf{v}_1$ Output directly during the forward pass.

$$\sigma_1 = [\mathbf{u}_1^\top]_{s, g} \mathbf{W} [\mathbf{v}_1]_{s, g} \quad (5)$$

in $[\cdot]_{s, g}$ It's a stopping gradient operator, so during automatic differentiation, it will only apply to the gradient. $\mathbf{W}^\top$ The result was exactly what was requested. $\mathbf{u}_1 \mathbf{v}_1^\top$ To prevent backpropagation $\mathbf{u}_1, \mathbf{v}_1$ The power iteration is internal.

No waste

pass T Through exponential iteration, we obtained the vector sequence. $\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(T)}$ But in the end we only used $\mathbf{v}^{(T)}$ To estimate σ_1 This seems rather "wasteful," so some people thought of making use of them to form a better estimate of the spectral norm.

Barring unforeseen circumstances, $\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(T)}$ They are different, but they will become more and more similar. $\mathbf{v}_1$ We can imagine them as " $\mathbf{v}_1$ Adding some kind of noise, and then performing "noise reduction" on them to gether, could indeed potentially yield more accurate results. Specifically, we consider... $\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(T)}$ A linear combination to give $\mathbf{v}_1$ To construct a better approximation, the subspace spanned by this set of vectors is called the "Krylov subspace".

For simplicity, we first perform orthogonalization to obtain an equivalent orthonormal basis.

$\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_T] \in \mathbb{R}^{m \times T}$ Next, we need to find the coefficients. $\mathbf{x} = [x_1, x_2, \dots, x_T]^\top \in \mathbb{R}^T$ making the vector $\sum_{i=1}^T x_i \mathbf{q}_i = \mathbf{Q} \mathbf{x}$ Try to satisfy

$$\mathbf{W}^\top \mathbf{W} \mathbf{Q} \mathbf{x} \approx \sigma_1^2 \mathbf{Q} \mathbf{x} \quad (6)$$

Multiply by both sides $\mathbf{Q}^\top$ have to $\mathbf{Q}^\top \mathbf{W}^\top \mathbf{W} \mathbf{Q} \mathbf{x} \approx \sigma_1^2 \mathbf{x}$ This indicates σ_1^2 , $\mathbf{x}$ They are $(\mathbf{W} \mathbf{Q})^\top \mathbf{W} \mathbf{Q}$ The eigenvalues and eigenvectors mean that we only need to perform eigenvalue decomposition on it, obtain its largest eigenvalue, and then take its square root to get a better estimate, since this is only a... $T \times T$ Matrix, when T When the size is small, performing eigenvalue decomposition is not a costly task, so existing functions can be called directly.

This approach is essentially a simplified version of the "Lanczos algorithm" in modern SVD solvers, although actual SVD implementations involve more sophisticated processing. Furthermore, this acceleration strategy shares similarities with Randomized SVD, which typically uses Gaussian random matrices for projection, while here we use an orthonormal basis in the Krylov subspace for projection.

Acceleration Skill

The following is a sample code for exponentiation acceleration using Krylov subspaces:

```

1 | @jax.jit(static_argnums=(1, 2))
2 | def spec_norm_v2(w, T=10, key=42):
3 |     v_shape = w.shape[:-2] + w.shape[-1:] + (1,)
4 |     v_init = jax.random.normal(jax.random.PRNGKey(key), v_shape)
5 |     v_step = lambda v, _: (l2_normalize(w.mT @ (w @ v)),) * 2
6 |     v = lax.scan(v_step, v_init, length=T)[1].swapaxes(0, -1)[0]
7 |     v = jnp.linalg.qr(v)[0]
8 |     return jnp.linalg.eigvalsh((u := w @ v).mT @ u)[..., -1]**0.5

```

Real-world testing shows that the subspace acceleration effect is very significant. $T = 5$ its accuracy has now surpassed that of the original power iteration. $T = 10$ it offers higher accuracy and faster speed. $T = 10$ At that time, about 2/3 of the time of this function comes from exponential iteration, 1/3 comes from QR decomposition, and the time spent on eigenvalue decomposition is very small.

Readers who have read the "Streaming Power Iteration" series might think of using Cholesky QR introduced there to accelerate QR decomposition. However, unfortunately, since the results of power iteration will get closer and closer to collinearity, the condition number of the QR matrix will continue to deteriorate. This is exactly the range where Cholesky QR is "powerless" and has an extremely high failure rate, so it is basically impossible to use it to accelerate the process.

A more effective acceleration method is to use only the results of the last few steps of the exponential iteration, such as the last three steps. $v^{(T-2)}, v^{(T-1)}, v^{(T)}$ We can construct a Krylov subspace to accelerate the process. This achieves most of the performance gains and reduces the computational cost of QR decomposition and eigenvalue decomposition, thus resulting in acceleration.

Estimate the upper limit

Power iteration and its accelerated version can, strictly speaking, only obtain the lower bound of the spectral norm, which is sufficient in many cases. However, in certain scenarios, we must obtain the upper bound, such as when we want to use spectral normalization to ensure that the norm of a matrix is strictly less than 1, then we need to find another method.

A basic approach at this point is to calculate the Schatten norm, which is precisely the upper bound of the spectral norm. Let the matrix... W SVD is $U\Sigma V^T$ singular value $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$ So its Schatten- p Norm is defined as

$$\|W\|_{S,p} = \sqrt[p]{\sum_{i=1}^n \sigma_i^p} \geq \sigma_1 \quad (7)$$

p The larger the value, the closer it is to the accurate value. When p When it is even, Schatten- p The calculation of the norm is relatively feasible, mainly by utilizing the following identity.

$$\| \mathbf{W} \|_{S2^k} = \sqrt[2^k]{\sum_{i=1}^m \sigma_i^{2^k}} = \sqrt[2^k]{\text{tr}(\mathbf{V} \Sigma^{2^k} \mathbf{V}^\top)} = \sqrt[2^k]{\text{tr}((\mathbf{W}^\top \mathbf{W})^k)} \quad (8)$$

$$\| \mathbf{W} \|_{S4k} = \sqrt[4k]{\sum_{i=1}^m \sigma_i^{4k}} = \sqrt[2^k]{\| \mathbf{V} \Sigma^{2^k} \mathbf{V}^\top \|_{S,2}} = \sqrt[2^k]{\| (\mathbf{W}^\top \mathbf{W})^k \|_{S,2}} \quad (9)$$

It is not difficult to see that Schatten-2 norm is actually the F-norm, which can be obtained by taking the square root of the sum of the squares of each element. Therefore, the two identities in the above equation show that as long as we obtain $(\mathbf{W}^\top \mathbf{W})^k$ then it can be obtained at a lower cost. $\| \mathbf{W} \|_{S2^k}$ and $\| \mathbf{W} \|_{S4k}$.

first step $\mathbf{W}^\top \mathbf{W}$ The complexity is $\mathcal{O}(nm^2)$ The subsequent repeated self-multiplication has a complexity of $\mathcal{O}(n \log n)$. $\mathcal{O}(m^3)$ So $\mathcal{O}(nm^2 + Tm^3)$ The complexity can be calculated. $(\mathbf{W}^\top \mathbf{W})^{2^T}$ Theoretically, this complexity far exceeds that of exponential iteration, even when n, m . When the size is relatively large, the first step is $\mathbf{W}^\top \mathbf{W}$ This goes far beyond power iteration. Therefore, if you only want to estimate the spectral norm and do not require an upper bound, you should still prioritize power iteration.

However, matrix multiplication can usually be performed in full parallelism, so when n, m Not big or $n \gg m$ At that time, Schatten- p If the norm doesn't present a computational bottleneck, then we can consider it. Alternatively, when we absolutely need a strict upper bound, this seems to be the most direct approach.

Maintain numerical value

In order to calculate $\| \mathbf{W} \|_{S,2^{T+2}}$ The simple realization is from $\mathbf{M} = \mathbf{W}^\top \mathbf{W}$ Set off, $\mathbf{M} \leftarrow \mathbf{M}^2$ Repeat execution T This time received $(\mathbf{W}^\top \mathbf{W})^{2^T}$ Then substitute into the formula (9) The calculation is performed, but because it involves super-exponential operations, it will quickly explode to NaN or collapse to zero.

Under normal circumstances, $\| \mathbf{W} \|_{S,2^{T+2}}$ The numbers themselves don't explode; the problem lies in explicit calculations. $(\mathbf{W}^\top \mathbf{W})^{2^T}$ The solution is to renormalize after each exponentiation step. $\mathbf{M} \leftarrow \mathbf{M}^2 / \text{tr}(\mathbf{M}^2)$ This prevents numerical explosion and ensures that the scaling is compact enough to avoid collapsing to zero; at the same time, we accumulate the normalization factor in the logarithmic field for the final calculation. $\| \mathbf{W} \|_{S,2^{T+2}}$ Used.

The calculation process is summarized as follows:

Calculate $\ \mathbf{W} \ _{S,2^{T+2}}$ As an upper bound of the spectral norm	
1 :	Initialize $\log S = \log \text{tr}(\mathbf{W}^\top \mathbf{W})$, $\mathbf{M} = \frac{\mathbf{W}^\top \mathbf{W}}{\text{tr}(\mathbf{W}^\top \mathbf{W})}$
2 :	For $t = 1, 2, \dots, T$ do
3 :	$\log S \leftarrow 2 \log S + \log \text{tr}(\mathbf{M}^2)$
4 :	$\mathbf{M} \leftarrow \frac{\mathbf{M}^2}{\text{tr}(\mathbf{M}^2)}$
5 :	Output $\exp\left(\frac{\log S + \log \ \mathbf{M} \ _F}{2^{T+1}}\right)$

A simple reference implementation:

```

1 | @jax.jit
2 | def tr(w):
3 |     return w.trace(axis1=-1, axis2=-2)[..., None, None]
4 |
5 | @jax.jit(static_argnums=(1,))
6 | def spec_norm_v3(w, T=5):
7 |     m = (m := w.mT @ w) / (s := tr(m))
8 |     ms_step = lambda i, ms: ((m := ms[0] @ ms[0]) / (s := tr(m)), 2 * ms[1] +
9 |     m, logs = lax.fori_loop(0, T, ms_step, (m, jnp.log(s)))
10 |    logf = 0.5 * jnp.log((m**2).sum(axis=[-1, -2], keepdims=True))
11 |    return jnp.exp((logs + logf) / 2**(T + 1))

```

Multi-order moments

In order to calculate $\|W\|_{S, 2^{T+2}}$ We need to calculate in some way $W^T W, \dots, (W^T W)^{2^{T-1}}, (W^T W)^{2^T}$ This means we can obtain at the same time $\|W\|_{S, 2}, \dots, \|W\|_{S, 2^{T+1}}, \|W\|_{S, 2^{T+2}}$ But in the end, only one was used. $\|W\|_{S, 2^{T+2}}$ That also seems somewhat "wasteful".

Is there a way to utilize these results to improve estimation accuracy, similar to the power-law iterative Krylov subspace method? Yes, there is! "Fast Tight Spectral-Norm Bounds" provides an approach to improve estimation results through nonlinear programming. Let's consider a simple case: suppose we obtain the second and fourth moments of the singular values.

$$\sum_{i=1}^m \sigma_i^2 = S_2, \quad \sum_{i=1}^m \sigma_i^4 = S_4 \quad (10)$$

Based on the results of the previous two sections... $S_4^{1/4}$ Compare $S_2^{1/2}$ Closer to the spectral norm, so we return $S_4^{1/4}$ As the upper bound of the spectral norm, S_2 It should be discarded and not used. But S_2 Is it really useless? I imagine if... $m = 3$ so, with exactly two equations and two unknowns, theoretically we can... σ_1, σ_2 Solve it! $m > 2$ While it's impossible to achieve an exact solution, it's still possible to narrow down the time. σ_1 The scope. Specifically, we have

$$\frac{S_2 - \sigma_1^2}{m - 1} = \frac{1}{m - 1} \sum_{i=2}^m \sigma_i^2 \leq \sqrt{\frac{1}{m - 1} \sum_{i=2}^m \sigma_i^4} = \sqrt{\frac{S_4 - \sigma_1^4}{m - 1}} \quad (11)$$

Right now $(S_2 - \sigma_1^2)^2 \leq (m - 1)(S_4 - \sigma_1^4)$ This is essentially a quadratic inequality in one variable, which is easy to solve.

$$\sigma_1 \leq \sqrt{\frac{S_2 + \sqrt{(m - 1)(mS_4 - S_2^2)}}{m}} \quad (12)$$

This gives us a way to use S_2 and S_4 . The representation σ_1 The upper limit is higher than $S_4^{1/4}$. A better estimate. *Fast Tight Spectral-Norm Bounds* also extends this to simultaneous use... S_2, S_4, S_6, S_8 . The form is more compact but also more cumbersome; interested readers can refer to the original text for further study. Using more moments to improve the estimation is theoretically feasible, but in practice, it often requires solving complex systems of nonlinear equations, making it less practical.

Limitations

Theoretically, the result (12) It can also be extended to use any S_{2k} and S_{4k} . To obtain a more accurate upper bound:

$$\sigma_1 \leq \sqrt{\frac{S_{2k} + \sqrt{(m-1)(m)S_{4k} - S_{2k}^2}}{m}} \quad (13)$$

However, when k When the value is large, the practical significance of this result is very limited because it requires explicit calculation. S_{2k} and S_{4k} . This is in k Larger structures would explode or collapse, which is impractical. One possible improvement is to... $S_{4k}^{1/4}$. Separate:

$$\sqrt{\frac{S_{2k} + \sqrt{(m-1)(m)S_{4k} - S_{2k}^2}}{m}} = S_{4k}^{1/4} \cdot \sqrt{\frac{S_{2k}/S_{4k}^{1/2} + \sqrt{(m-1)(m - S_{2k}^2/S_{4k})}}{m}} \quad (14)$$

Then set $S_{2k}/S_{4k}^{1/2} = e^\epsilon$. In the approximate expansion of the number field

$$\sqrt{\frac{S_{2k}/S_{4k}^{1/2} + \sqrt{(m-1)(m - S_{2k}^2/S_{4k})}}{m}} \approx 1 - \frac{\epsilon^2}{4k(m-1)} \quad (15)$$

However, our goal is to find the upper bound of the spectral norm, and ensuring this upper bound property while performing an approximate expansion seems rather complex. On the other hand, if we have already calculated a relatively large one... k Actually $S_{4k}^{1/4}$. The accuracy is already quite high, so there's not much point in using planning concepts to further improve the precision.

End of article

This article briefly summarizes several approaches to estimating the spectral norm. In practical applications, if only approximate monitoring of the spectral norm is required, power iteration and its subspace accelerated version are usually sufficient; if a strict upper bound must be guaranteed, the Schatten norm can be considered. The "no-waste" ideas derived from these two approaches—Krylov subspace utilizing iteration history and nonlinear programming utilizing multi-moment information—also provide excellent inspiration for algorithm design.

Please include this article's URL when reprinting: <https://kexue.fm/archives/11736>

For more detailed information on reprinting, please refer to: "Science Space FAQ"

If you need to cite this article, please refer to:

Su Jianlin. (May 04, 2026). "How to Estimate the Spectral Norm of a Matrix More Scientifically?" [Blog post]. Retrieved from <https://kexue.fm/archives/11736>

```
@online{kexuefm-11736,  
  title={How to Estimate the Spectral Norm of a Matrix More Scientifically?},  
  author={苏剑林},  
  year={2026},  
  month={May},  
  url={\url{https://kexue.fm/archives/11736}},  
}
```